AUGMENTED DECISION · FINAL PROJECT REPORT

Flagging bankruptcy *before* it happens

Cost-sensitive screening of Taiwanese firms with LASSO selection and a decision tree

Yassine ERRAJI · **William PELTIER** · **Ben DAESH** · **Simon AMAR**

23 October 2026 · Code: github.com/yassineerraji/Augmented-Decision-Final-Project

ABSTRACT

A credit analyst must decide whether to flag a listed firm as high-risk using its financial ratios. Bankruptcies are rare (3.2 % of 6,819 Taiwanese firms) and a missed one is assumed to cost ten false alarms, so models are judged on the *average misclassification cost per firm*, and a firm is flagged when its estimated bankruptcy probability reaches the Bayes threshold $1/11$. We select variables with an L1-penalised (LASSO) logistic regression, refit inside every cross-validation fold, and feed the 43 selected ratios to a depth-4 decision tree. On 1,364 held-out firms the final model costs **0.179** per firm against **0.323** for not screening (-45 %; 95 % bootstrap interval of the gain $[0.065, 0.228]$), catching 32 of the 44 bankruptcies. The tree uses only 8 ratios and reads as a handful of rules; a random forest would be about 10 % cheaper in cross-validation, the price we pay for interpretability.

1 Problem formulation

Decision and target. A credit analyst reviews a listed firm and chooses between two actions: *flag* it as high-risk (deny or reprice credit, open a closer review) or *do not flag* it. The target is $Y =$ `Bankrupt?`, equal to 1 if the firm went bankrupt under the Taiwan Stock Exchange rules. The inputs x are 95 financial ratios, assumed to be known before the outcome.

Loss and decision rule. The two possible errors do not cost the same (Table 1). With $p = P(Y = 1 \mid x)$, flagging has expected cost $C_{FP}(1 - p)$ and not flagging $C_{FN}\,p$, so the Bayes decision rule (Lecture 1) flags a firm when

$$p \geq \frac{C_{FP}}{C_{FP} + C_{FN}} = \frac{1}{11} \approx 0.09. \tag{1}$$

The costs $C_{FN} = 10$ and $C_{FP} = 1$ are stipulated, not estimated; Section 3.4 shows how the decisions change with the cost ratio.

Table 1. Loss table of the decision problem.

	Healthy firm ($Y = 0$)	Bankrupt firm ($Y = 1$)
Do not flag ($a = 0$)	0	$C_{FN} = 10$ (missed bankruptcy)
Flag ($a = 1$)	$C_{FP} = 1$ (false alarm)	0

Metric and baseline. A model is scored by its empirical risk under the loss table, the *average cost per firm* (lower is better), complemented by recall and precision:

$$\hat{R}_n = \frac{1}{n}\sum_{i=1}^{n} L(\hat{a}_i, y_i) = \frac{C_{FN}\cdot \mathrm{FN} + C_{FP}\cdot \mathrm{FP}}{n}. \tag{2}$$

Accuracy is unsuitable: the majority-class rule "never flag" is 96.8 % accurate while catching no bankruptcy. That rule is our **baseline**; its cost is $10 \times 0.032 \approx 0.323$ per firm.

Data. The data come from the Taiwan Economic Journal for 1999–2009 and are distributed by the UCI Machine Learning Repository [1, 2]. Table 2 summarises them. Two features of the data shape our method. First, the ratios are highly redundant: 50 pairs have a Spearman correlation $|\rho| > 0.95$, involving 47 of the 94 usable ratios (Figure 1). Second, 24 ratios mix values in $[0, 1]$ with values between 6×10^5 and 10^{10}, with nothing in between, which looks like an encoding artefact. The strongest single predictors are profitability ratios (EPS, net income to total assets, ROA), each with a univariate AUC between 0.86 and 0.89. This exploratory analysis is descriptive only; every modelling choice below is made on the training set.

Table 2. The dataset at a glance.

Observations	6,819 listed Taiwanese firms, 1999–2009
Variables	95 financial ratios (one, `Net Income Flag`, is constant and is dropped: 94 remain)
Target distribution	220 bankrupt firms (3.2 %), 6,599 healthy firms
Missing values, duplicates	none, none
Preprocessing required	drop the constant column; handle 24 ratios with extreme values (Section 2.3)

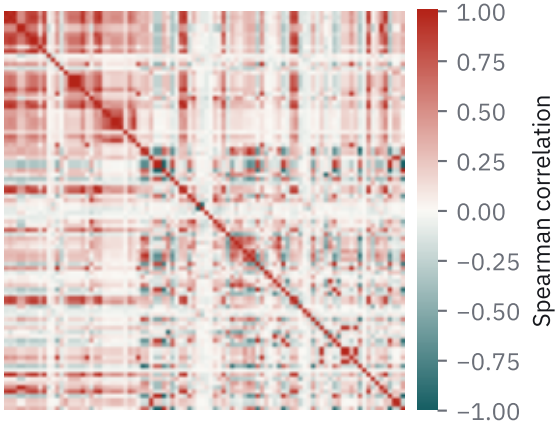


Figure 1. Spearman correlation matrix of the 94 ratios, in dataset column order. Blocks of near-duplicates (profitability, per-share values, growth, leverage, cash flow) motivate variable selection.

2 Methods and choices

2.1 Overview

Figure 2 summarises the approach ("variable selection with decision-tree classification"). A LASSO logistic regression chooses a subset of ratios; a decision tree, grown on the raw values

of those ratios, estimates the bankruptcy probability in each leaf; the Bayes rule (1) turns that probability into a decision.

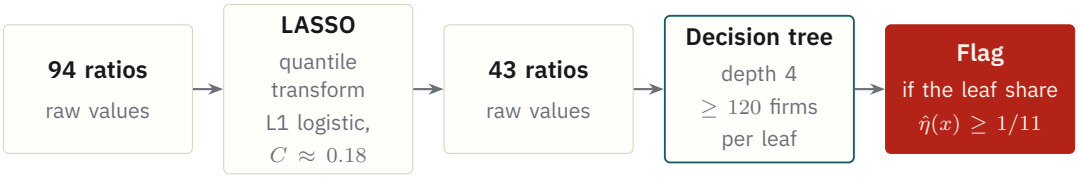


Figure 2. The final pipeline. Everything left of the decision is refit inside every cross-validation fold.

2.2 Data split and validation protocol

We hold out a stratified 20 % test set (1,364 firms, 44 bankrupt) with a fixed seed; the remaining 5,455 firms (176 bankrupt) form the training set. The test set is opened once, at the end, for the frozen final model and the baseline only (Lecture 2). All tuning and model comparisons use the *same* stratified 5-fold cross-validation on the training set. Preprocessing and selection are part of scikit-learn pipelines [3], so in every fold the quantile transform, the LASSO and the tree are refit on four folds and scored on the fifth: no information leaks from the validation fold. Code 1 shows the metric used throughout.

Code 1. The decision cost as a scikit-learn scorer (notebook, Section 3.1).

```
def avg_cost(y_true, y_pred, c_fn=C_FN, c_fp=C_FP):
    return np.mean(c_fn * ((y_true == 1) & (y_pred == 0))
                   + c_fp * ((y_true == 0) & (y_pred == 1)))

cost_scorer = make_scorer(avg_cost, greater_is_better=False)
```

2.3 Preprocessing

The constant column is dropped. The 24 ratios with extreme values are *not* capped: among the 4,816 training firms whose `Total Asset Growth Rate` exceeds 1, the value alone still ranks bankruptcies with an AUC of 0.72, so the extreme values carry signal. The two models need different treatments. The tree only uses the order of the values and receives the raw ratios. The LASSO is sensitive to scale, so it receives a rank-based quantile transform to a standard normal, fitted on the training folds only: it keeps the order of the values, removes the magnitude problem and puts all ratios on a common scale, which a fair L1 penalty requires.

2.4 Variable selection: LASSO logistic regression

For a binary outcome, the LASSO of Lecture 4 becomes an L1-penalised logistic regression with an unpenalised intercept:

$$\min_{b_0,\,\beta} \; -\frac{1}{n}\sum_{i=1}^{n}\Big[y_i \log p_i + (1-y_i)\log(1-p_i)\Big] + \lambda\|\beta\|_1, \qquad p_i = \frac{1}{1+e^{-(b_0+x_i^{\top}\beta)}}. \tag{3}$$

The L1 penalty sets many coefficients exactly to zero; the corresponding ratios are dropped. In scikit-learn the penalty is set through $C = 1/(n\lambda)$. We prefer the LASSO to forward selection (Lecture 3.2) because a single parameter controls the number of variables, it copes with

the redundant ratios, and one fit takes under a second, so it can be refit inside every fold; forward selection would need up to 94 fits per added variable, repeated in every fold. The intercept is kept unpenalised as in the course (`intercept_scaling=100` makes `liblinear`'s intercept penalty negligible), and no class weights are used: the LASSO only chooses variables, and the costs enter at the decision stage.

Penalty. C is chosen by 5-fold cross-validation on the training set, minimising the log-loss, a proper scoring rule and the classification counterpart of the validation MSE used in the course, over 13 values from 10^{-3} to 1 (Code 2). The minimum, $C \approx 0.178$, keeps **43 of the 94 ratios**, with a CV log-loss of 0.0855 against 0.143 with no variable (Figure 3). The curve is flat: every C from 0.056 (21 ratios) to 0.562 (59 ratios) is within one standard error of the minimum.

Code 2. LASSO pipeline and choice of the penalty (notebook, Sections 2.1–2.2).

```
lasso = make_pipeline(QuantileTransformer(output_distribution="normal"),
                      LogisticRegression(l1_ratio=1, solver="liblinear",
                                         intercept_scaling=100, random_state=SEED))
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)
C_grid = np.logspace(-3, 0, 13)
lasso_cv = GridSearchCV(lasso, {"logisticregression__C": C_grid}, cv=cv,
                        scoring="neg_log_loss", n_jobs=-1).fit(X_train, y_train)
```

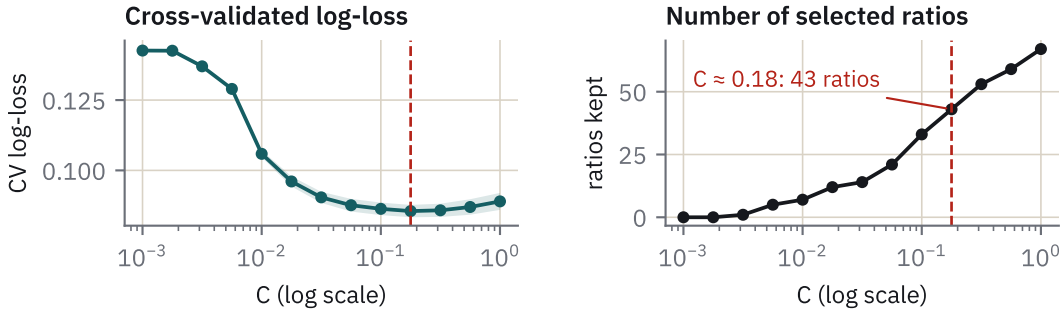


Figure 3. Choice of the LASSO penalty. Left: cross-validated log-loss (mean $\pm$ 1 SE). Right: number of ratios kept when refit on the full training set. Dashed line: chosen $C \approx 0.18$.

What is selected, and how stable it is. The leading coefficients (Table 3) cover leverage, profitability and liquidity, with intuitive signs: more debt raises the predicted risk, more profit and liquidity lower it. Following Lecture 4, we refit the whole procedure on 100 stratified bootstrap resamples of the training set (Figure 4). Only 15 of the 43 selected ratios are kept in at least 80 % of the resamples, and some ratios left out are often selected, such as the R&D expense rate (81 %): correlated ratios swap places. The selected set is a good predictive shortlist, not a list of causes.

2.5 Decision tree and decision rule

A classification tree (CART, Gini impurity $G(q) = 2q(1-q)$, Lecture 3) is grown on the raw values of the selected ratios. The share of bankrupt training firms in a leaf estimates $\hat{\eta}(x) = P(Y = 1 \mid x)$ for every firm reaching it, and the firm is flagged when $\hat{\eta}(x) \geq 1/11$. The costs therefore enter through the *threshold*, not through class weights: this keeps the leaves' reading as probabilities

Table 3. The eight largest LASSO coefficients at $C \approx 0.178$ (quantile-normal scale, so magnitudes are comparable).

Ratio	Coefficient	Ratio	Coefficient
Total debt / total net worth	$+0.819$	Quick ratio	-0.320
ROA(B) before interest	-0.477	Working capital turnover	$+0.314$
Retained earnings / total assets	-0.415	Persistent EPS (4 seasons)	-0.311
Interest-bearing debt interest rate	$+0.350$	Borrowing dependency	$+0.308$

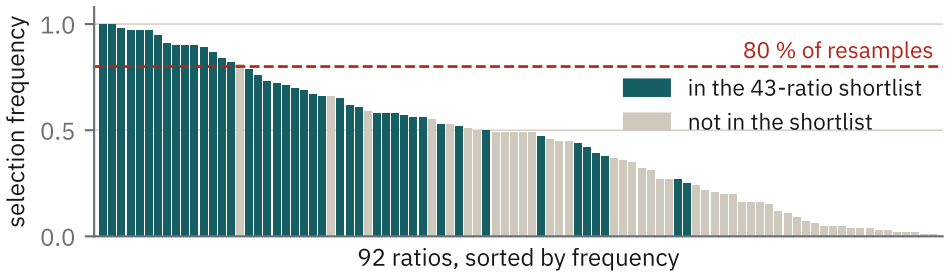


Figure 4. Selection stability: share of 100 bootstrap refits in which each ratio is selected (92 ratios selected at least once). Teal: in the 43-ratio shortlist; grey: not in it.

and lets us test other cost ratios without retraining. In code, the whole pipeline is wrapped in a `FixedThresholdClassifier`, so its predictions are directly the decisions (Code 3).

Hyperparameters. We tune the maximum depth, $\{2, 3, 4, 5, 6, 8, \text{no limit}\}$, and the minimum number of firms per leaf, $\{1, 10, 20, 40, 80, 120, 160, 200\}$, on the decision cost. Large leaves are needed for the leaf shares to be reliable probabilities: with a 3.2 % bankruptcy rate, a leaf of 120 firms holds only about four bankruptcies. Figure 5 shows the effect. With leaves of one firm, the training cost falls to zero as the tree deepens while the CV cost climbs from 0.17 to 0.23: the tree memorises the training firms (Lecture 2). With at least 120 firms per leaf, the gap stays small and depth 4 is best (0.151).

Code 3. Final pipeline M2 and its tuning (notebook, Sections 2.4 and 3.3).

```
selector = SelectFromModel(lasso.set_params(logisticregression__C=best_C),
                           importance_getter="named_steps.logisticregression.coef_",
                           threshold=1e-5)  # keep the non-zero coefficients
pipe = FixedThresholdClassifier(
    Pipeline([("select", selector),
              ("tree", DecisionTreeClassifier(random_state=SEED))]),
    threshold=THRESHOLD)                    # flag when P(bankrupt | x) >= 1/11
tree_grid = {"max_depth": [2, 3, 4, 5, 6, 8, None],
             "min_samples_leaf": [1, 10, 20, 40, 80, 120, 160, 200]}
m2_grid = {"estimator__select__estimator__logisticregression__C": [best_C],
           **{f"estimator__tree__{k}": v for k, v in tree_grid.items()}}
m2 = GridSearchCV(pipe, m2_grid, cv=cv, scoring=cost_scorer, n_jobs=-1,
                  return_train_score=True).fit(X_train, y_train)
```

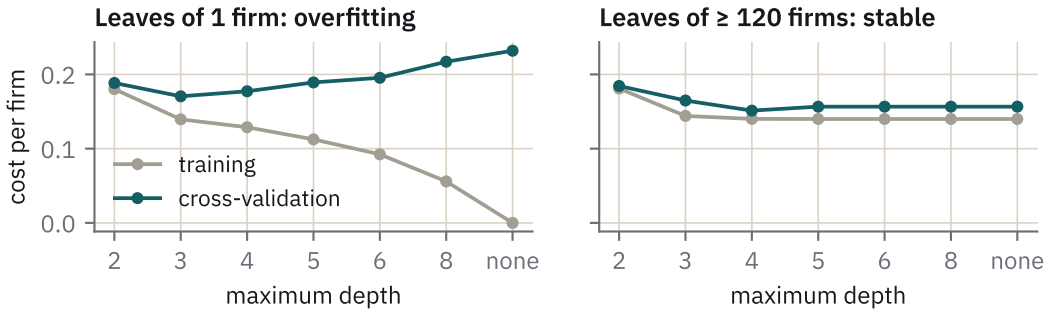


Figure 5. Validation curves of the final pipeline: training and cross-validated cost per firm against the maximum depth, for leaves of one firm (left) and of at least 120 firms (right).

3 Evaluation

3.1 Cross-validated model selection

We compare four candidates on the same folds: **M0**, the baseline; **M1**, a tuned tree on all 94 ratios; **M2**, the LASSO selector with C fixed at its Section 2.4 value, followed by a tuned tree; and **M3**, the same pipeline with C tuned jointly with the tree over the seven grid values from 0.01 to 0.316. The selection rule was fixed before seeing these results: M2, the course protocol, is the reference, and an alternative replaces it only if its CV cost is lower by more than one standard error. Because all candidates share the folds, the comparison is *paired*: for each fold k we compute the gain $\Delta_k = \text{cost}_k(\text{M2}) - \text{cost}_k(\text{alternative})$ and its standard error $s_\Delta/\sqrt{5}$ (Code 4).

Table 4. Cross-validated results (5 folds, training set). Paired gain of each alternative over M2: mean $\pm$ standard error, and number of folds in which it is cheaper than M2.

Candidate	CV cost	Depth	Leaf	C	Ratios	Paired gain over M2
M0 Never flag	0.323	–	–	–	0	-0.171 ± 0.009 (0/5)
M1 Tree, all ratios	0.163	4	120	–	94	-0.012 ± 0.005 (0/5)
M2 LASSO + tree	**0.151**	4	120	0.178	43	reference
M3 LASSO (joint C) + tree	0.148	3	10	0.01	7	$+0.003 \pm 0.007$ (3/5)
M4 LASSO + random forest	0.136	–	10	0.178	43	$+0.015 \pm 0.007$ (5/5)

Code 4. Paired selection rule (notebook, Section 3.4).

```
# per-fold gain of each candidate over M2 (> 0: cheaper than M2)
gains = fold_costs.rsub(fold_costs["M2 LASSO, C fixed"], axis=0)
rule = pd.DataFrame({"mean gain over M2": gains.mean(),
                     "paired SE": gains.std() / np.sqrt(5),
                     "folds cheaper than M2": (gains > 0).sum()})
rule["replaces M2"] = rule["mean gain over M2"] > rule["paired SE"]
```

Table 4 and Figure 6 give the results. Every tree halves the cost of the baseline. **Variable selection helps**: M2 beats the tree on all 94 ratios by 0.012 ± 0.005 and is cheaper or tied on every fold. Joint tuning (M3) is not significantly better (0.003 ± 0.007), although it shows that a 7-ratio selection works about as well. M2 is therefore the final model.

Benchmark: the price of interpretability. To measure what a single readable tree costs, we also cross-validated a random forest of 300 trees on the LASSO-selected ratios (M4), tuning its leaf size. It is a benchmark, not a candidate: the brief asks for a decision-tree classifier and our recommendation relies on reading the tree's rules, so M4 is never scored on the test set. It is about 10 % cheaper (0.015 ± 0.007, cheaper on all five folds), because averaging 300 trees gives much finer probabilities than nine leaves.

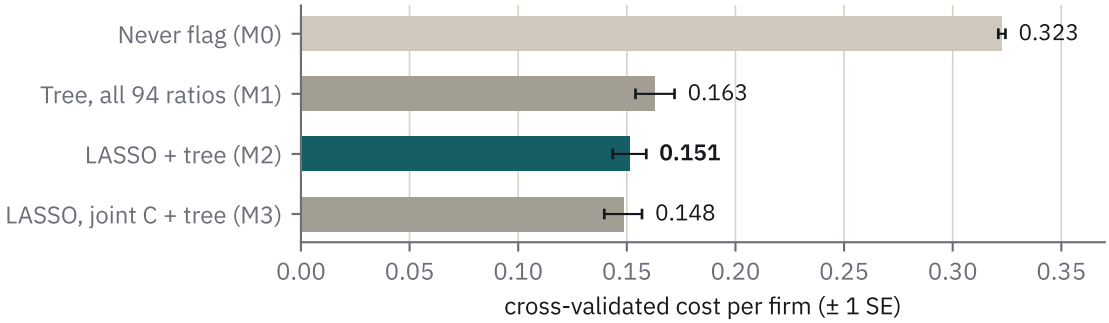


Figure 6. Cross-validated cost per firm of the four candidates ($\pm$ 1 standard error across folds).

3.2 The final model

Refit on the full training set, the LASSO keeps 43 ratios and the tree, of depth 4, has **9 leaves and uses only 8 of them** (Figure 7): the tree performs a second, finer selection.

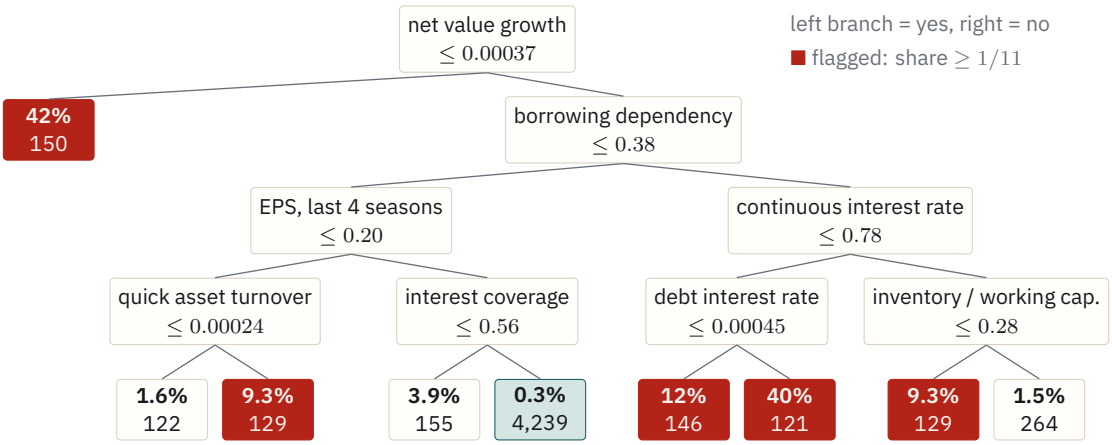


Figure 7. The final tree. Each leaf shows its share of bankrupt training firms and its number of firms; red leaves are flagged.

3.3 Test-set performance

The frozen model and the baseline are scored once on the 1,364 test firms (Table 5, Code 5). The model **costs 0.179 per firm against 0.323**, a 45 % reduction. It catches 32 of the 44 bankruptcies (recall 73 %) at the price of 124 false alarms (precision 21 %): 156 firms are flagged, about one in nine. Its accuracy (90 %) is lower than the baseline's (97 %), which confirms that accuracy would have rewarded the useless rule.

Uncertainty. With a loss bounded in $[0, C_{FN}]$ and $m = 1{,}364$ test firms, Lecture 2's Hoeffding bound gives a 95 % half-width of $C_{FN}\sqrt{\log(2/\alpha)/(2m)} = 0.37$, wider than the gap between the two rules: valid, but uninformative. We therefore use a 95 % percentile bootstrap over the test

Table 5. Test-set performance (1,364 firms, 44 bankrupt). Right: confusion matrix of the final model.

	Final model	Never flag
Average cost per firm	**0.179**	0.323
Recall	0.727	0.000
Precision	0.205	–
Accuracy	0.900	0.968
Firms flagged	156	0

	not flagged	flagged
healthy	1,196	124 false alarms
bankrupt	12 missed	32 caught

Code 5. Test predictions and bootstrap of the test costs (notebook, Sections 4.1–4.2).

```
y_pred = final_model.predict(X_test)        # decisions at the 1/11 threshold
y_base = np.zeros(len(y_test), dtype=int)   # "never flag"
boot = [resample(np.arange(len(y_test)), random_state=b) for b in range(1000)]
boot_costs = pd.DataFrame(
    [{"Final model": avg_cost(y_test.values[i], y_pred[i]),
      "Baseline": avg_cost(y_test.values[i], y_base[i])} for i in boot])
```

firms, with the predictions fixed and without stratification, so that the number of bankruptcies varies as in a new sample (Figure 8). The paired difference lies in $[-0.228, -0.065]$, entirely below zero: the model's advantage is not an artefact of this test set. The test cost is higher than the CV estimate (0.151), which nevertheless lies inside the model's interval; the gap is expected, since the test cost is noisy with only 44 bankruptcies and the CV cost of the chosen configuration, the best of 56, is slightly optimistic.

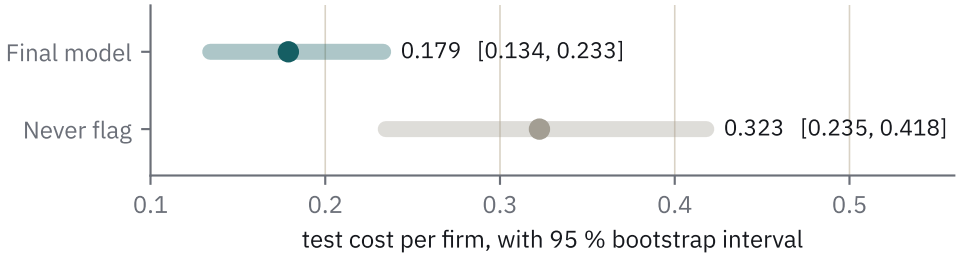


Figure 8. Test cost per firm with 95 % bootstrap intervals (1,000 resamples of the test firms). Gain of the model over the baseline: $[0.065, 0.228]$ per firm.

3.4 Sensitivity to the cost ratio

The 10:1 ratio is an assumption. For a ratio $r = C_{FN}/C_{FP}$ the Bayes threshold becomes $1/(1+r)$; since the tree is not retrained, only the threshold changes (Table 6). The model beats the baseline under every ratio, cutting its cost by 35 %, 45 % and 59 %. Each leaf is flagged once r exceeds its *switching ratio*

$$r^* = \frac{1-\hat{\eta}}{\hat{\eta}}, \qquad \text{since } \hat{\eta} \geq \frac{1}{1+r} \iff r \geq r^*. \tag{4}$$

Figure 9 shows that the two riskiest leaves (42 % and 40 %) are flagged for any ratio above 1.5, while two leaves (9.3 %) switch at 9.75, just below our assumed ratio. No leaf switches between 9.75 and 24.8, which is why ratios 10 and 20 give identical decisions.

Table 6. Test-set results under three cost ratios (tree unchanged, threshold $1/(1+r)$).

Cost ratio r	Threshold	Model cost	Baseline cost	Recall	Precision	Firms flagged
5	0.167	0.106	0.161	0.545	0.353	68
10	**0.091**	**0.179**	**0.323**	**0.727**	**0.205**	**156**
20	0.048	0.267	0.645	0.727	0.205	156

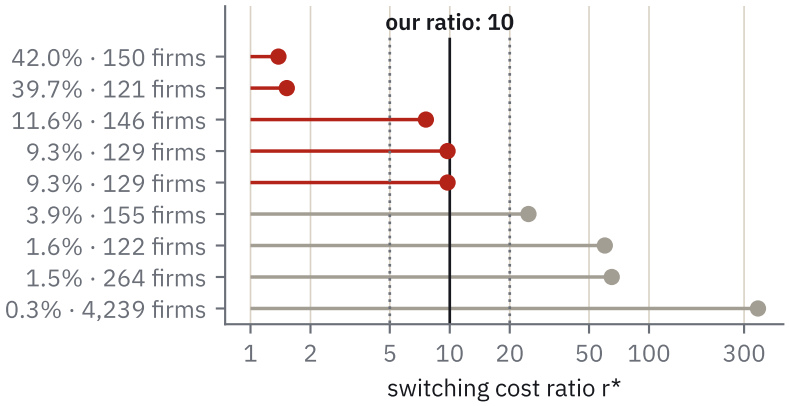


Figure 9. Switching cost ratio of each leaf (training share of bankrupt firms and number of firms). Red: flagged at our ratio of 10; dotted lines: ratios 5 and 20.

4 Evidence and recommendation

4.1 What the tree says

Reading the flagged leaves of Figure 7, the model raises an alarm in three situations: (i) the 2.7 % of firms with the weakest net value growth, of which 42 % went bankrupt; (ii) heavy reliance on borrowing combined with weak profitability, 40 % bankrupt when the debt is expensive and 12 % otherwise; (iii) two narrower pockets at 9.3 %, flagged only just. Conversely, firms with moderate borrowing, EPS above 0.20 and an interest coverage ratio above 0.56 hold 78 % of the training firms and a bankruptcy share of 0.3 %. These are predictive associations, not causal effects.

4.2 Recommendation

Following the decision-memo structure of Lecture 1:

1. **Action and information.** Flag every firm whose leaf has a bankruptcy share of at least $1/11$, using 8 financial ratios read by a depth-4 tree.
2. **Assumptions.** A missed bankruptcy costs ten false alarms, and the leaf shares of 5,455 training firms are taken as bankruptcy probabilities.
3. **Comparison with the alternative.** On unseen firms the model costs 0.179 per firm against 0.323 without screening, catching 32 of 44 bankruptcies for 124 false alarms. A random forest would lower the cross-validated cost by about 10 %, but without the readable rules this recommendation relies on.
4. **Conditions that would change the choice.** With a cost ratio below 9.75, two leaves would no longer be flagged, and below 7.6 a third; any ratio between 9.75 and 24.8 leaves the decisions unchanged.

4.3 Limitations

- **Stipulated costs.** The 10:1 ratio is an assumption, and two leaves depend on it being at least 9.75.
- **No time dimension.** Firms are pooled over 1999–2009 without a year, so the evaluation is a random split, not a forecast of future years; economic conditions may shift (Lecture 2).
- **Possibly repeated firms.** The rows may include several years of the same firm; a random split can then place one firm in both training and test, making the test cost optimistic. Without a firm identifier this cannot be checked.
- **Few bankruptcies.** With 176 bankruptcies in training and 44 in test, CV standard errors are about 0.008, the test interval is wide, and leaf shares rest on 2 to 63 bankruptcies. Nodes 6 and 15 are flagged with a share of 0.093, only 0.002 above the threshold, while that share's standard error is about $\sqrt{0.093 \times 0.907/129} \approx 0.026$.
- **No nested cross-validation.** M2 and M3 were compared on the folds used to tune them; the pre-registered rule limits but does not remove this optimism.
- **Unstable variable list.** Only 15 of the 43 LASSO ratios are robustly selected, and a 7-ratio selection works about as well.
- **Data quality.** Three of the eight ratios used by the tree contain extreme values that look like encoding artefacts; the order-based treatment limits, but may not remove, their impact.

Before deployment, the frozen procedure should be refit on all 6,819 firms and validated on more recent data.

5 Reproducibility

All results come from a single notebook, `src/bankruptcy_prediction.ipynb`, executed top to bottom with fixed seeds (`SEED = 42`); it runs in about 4.5 minutes with Python 3.12 and the pinned versions of `requirements.txt` (scikit-learn 1.9.1, pandas 3.0.6). The notebook stops with an explicit message if scikit-learn is older than 1.8, because earlier versions silently replace the L1 penalty. A script, `slides/make_figures.py`, exports the figures of this report from that same notebook.

```
uv venv --python 3.12 && uv pip install -r requirements.txt
.venv/bin/jupyter nbconvert --to notebook --execute --inplace \
    src/bankruptcy_prediction.ipynb
```

References

[1] UCI Machine Learning Repository (2020). *Taiwanese Bankruptcy Prediction*. doi:10.24432/C5004D.

[2] D. Liang, C.-C. Lu, C.-F. Tsai and G.-A. Shih (2016). Financial ratios and corporate governance indicators in bankruptcy prediction: a comprehensive study. *European Journal of Operational Research*, 252(2), 561–572.

[3] F. Pedregosa et al. (2011). Scikit-learn: machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

[4] Augmented Decision, lecture decks 1–4: decision theory, statistical learning, decision trees, variable selection and regularization.

A LLM assistance

The project was carried out with the help of an LLM, Claude Opus 5.5 (Anthropic), used through Claude Code in Visual Studio Code. The collaboration followed a protocol set out in `INSTRUCTIONS.md` and was documented in the `AgentDiscussion` folder, both described below.

A.1 The protocol: `INSTRUCTIONS.md`

This file was written before any code was produced. It defines the deliverable, the coding rules and the step-by-step workflow, and is reproduced in full.

`INSTRUCTIONS.md` full file

We are working on the Taiwanese Bankruptcy Prediction

– (https://archive.ics.uci.edu/dataset/572/taiwanese+bankruptcy+prediction), UCI Machine Learning Repository. Collected from the Taiwan Economic Journal, 1999–2009.

Our goal is to build a 4-step project that goes as follows:

1. Problem Formulation & Exploratory Data Analysis
2. Variable Selection
3. Model Selection and Decision Tree Tuning
4. Evaluation of the final model

Our end-deliverable is one notebook file (ipynb), containing the code that implements the 4 steps.
IMPORTANT: WE MUST CODE IN THE MOST SIMPLE WAY POSSIBLE.
WHAT IS MEANT BY SIMPLE IS:

– A python cell only does one task at a time.
– We always privilege code minimality, what can be done in two lines of codes must be done in no more than two lines, always.
– We always privilege simplicity and correctness over complexity and long code that is hard to visually understand.
– We always include comments in the beginning of a Python cell to explain what is being done.
– All the steps we code must build on top of the previous ones.
– When doing assumptions, or making decisions, they must never be done silently and appear in the form of concise markdowns cells.
– Markdown cells must follow a clean and coherent structure all the way through the notebook.

In the context folder you will find two elements:

1. The lecture decks of the course that will serve as great context to scope well the work we are doing.
2. The official instruction file for the project, provided by our teacher. Feel free to read it to have context on the final submission.

The workflow will be as follows:

We will be working step by step and not one-shot the full project.

1. You will explain very concisely your thoughts, assumptions and rationale in a folder called AgentDiscussion, where you will add a new markdown file at the beginning of each of the 4 steps of the project.
2. I will review in details the markdown files and either leave them as is or bring modification to them.

3. You will check the last state of the markdown cell and build according to it.
4. After you end coding, write a recap markdown file in the same AgentDiscussion folder and wait for me to review the code and the markdown file before signaling that we can proceed with the next step of the project.

A.2 The plan–recap workflow

Following `INSTRUCTIONS.md`, each of the four steps ran through the same loop (Figure 10). At the start of a step, the LLM wrote a **plan**, `stepN_plan.md`: its rationale and assumptions, the decisions to be recorded in the notebook, the planned cells, and a closing list of points to confirm or change. The plan was reviewed and edited, its open points were settled, and the resulting decisions were recorded in it. The LLM then re-read the final plan and built the step in the notebook. Once the step was coded, it wrote a **recap**, `stepN_recap.md`: what was built, the key results, every difference from the plan, and what carries over to the next step. The next step started only after the code and the recap had been reviewed. The `AgentDiscussion` folder holds one sub-folder per step (`step1/` to `step4/`), each with its plan and its recap.

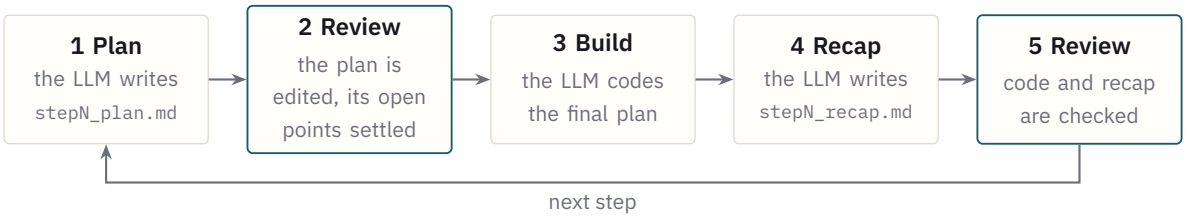


Figure 10. The plan–recap loop defined in `INSTRUCTIONS.md`, run once for each of the four steps.

A.3 Excerpts

The three excerpts below show the loop at work: the open points that close a plan, as edited during its review (Step 1); the decisions then recorded in a plan (Step 3); and the differences from the plan reported in a recap (Step 3).

`AgentDiscussion/step1/step1_plan.md` excerpt, plan: open points, as edited during review

5. Points for you to confirm or change

- **Cost ratio C_FN : C_FP = 10 : 1.** Alternatives: about 30 : 1, which roughly corresponds to balancing the two classes, or a non-cost metric such as F1 or recall at a fixed precision.
- **Test size of 20%.** Moving to 25% gives about 55 bankrupt firms in the test set instead of 44.
- **EDA on the full dataset.

`AgentDiscussion/step3/step3_plan.md` excerpt, plan: decisions recorded after review

6. Points for you to confirm or change

Decided (you approved the recommendations):

- The costs enter through the threshold.
- The grid stays as listed in decision 2.
- The agreed rule stays as it is, with no tie-breaker for fewer variables.

The original points are kept below for reference.

- **How costs enter:** through the threshold (recommended), or through class weights.
- **The grid in decision 2.**
- **Keeping the agreed rule, even though M3 reaches the same cost with about 7 variables.** The alternative is to add a tie-breaker: "within one standard error, prefer fewer selected variables", which would pick M3. I recommend keeping the agreed rule. It was set before seeing these results, and changing a rule after seeing the results weakens it.

`AgentDiscussion/step3/step3_recap.md` excerpt, recap: what changed while coding

Differences from the plan

1. **M2's grid states `C` explicitly** (`[best_C]`) instead of relying on the value preset inside `selector`. This is more robust and makes M2 and M3 directly parallel, since M3 just swaps in `C_grid[4:11]`.
2. **"Apply the rule and keep the final model" became two cells** (the rule table, then the freeze), to keep one task per cell. That is why there are 13 code cells instead of 12.
3. **Section 3.1 shows the metric as an empirical-risk formula**, and `avg_cost` computes exactly that: the mean of the per-firm losses.
4. **Two pandas 3 adjustments:**
   - In the validation-curve plot, depth "None" is converted to the label "no limit" before plotting.
   - The comparison table uses `convert_dtypes()`, so depths and leaf sizes display as whole numbers.